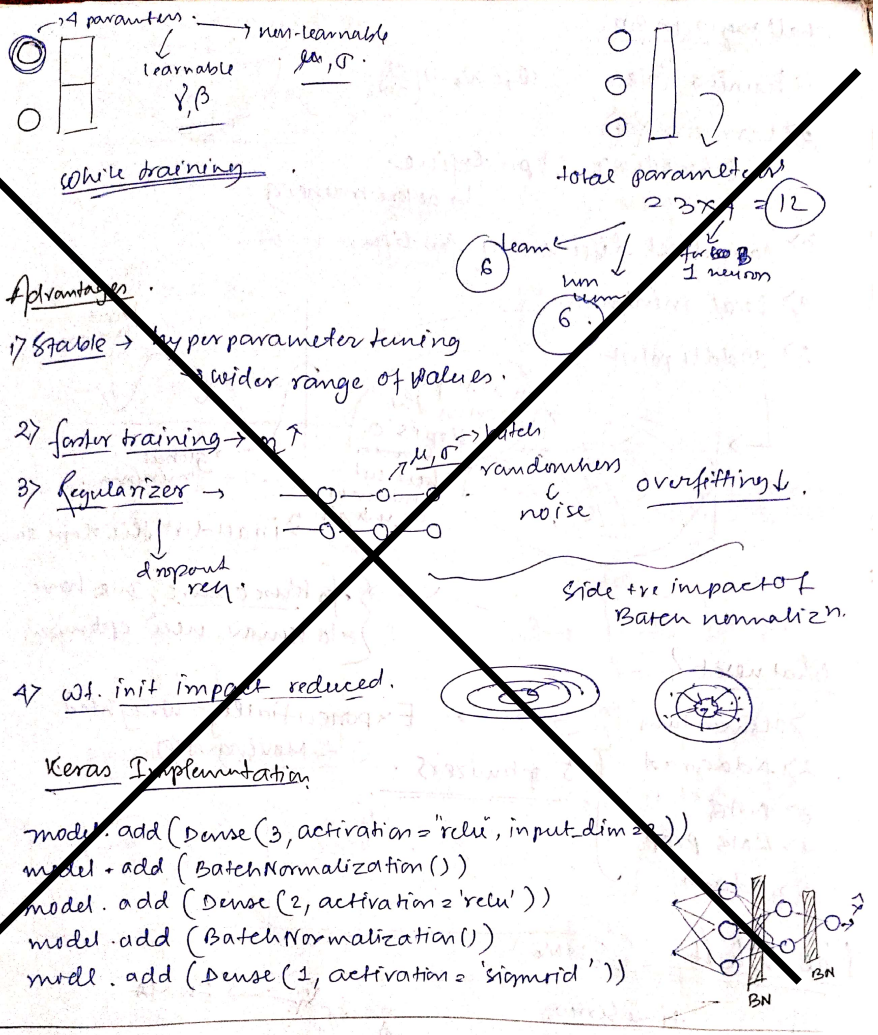
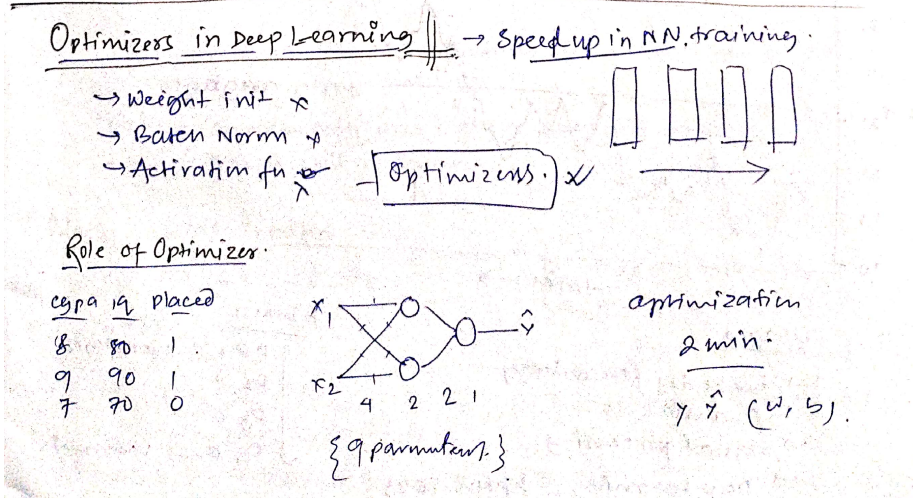
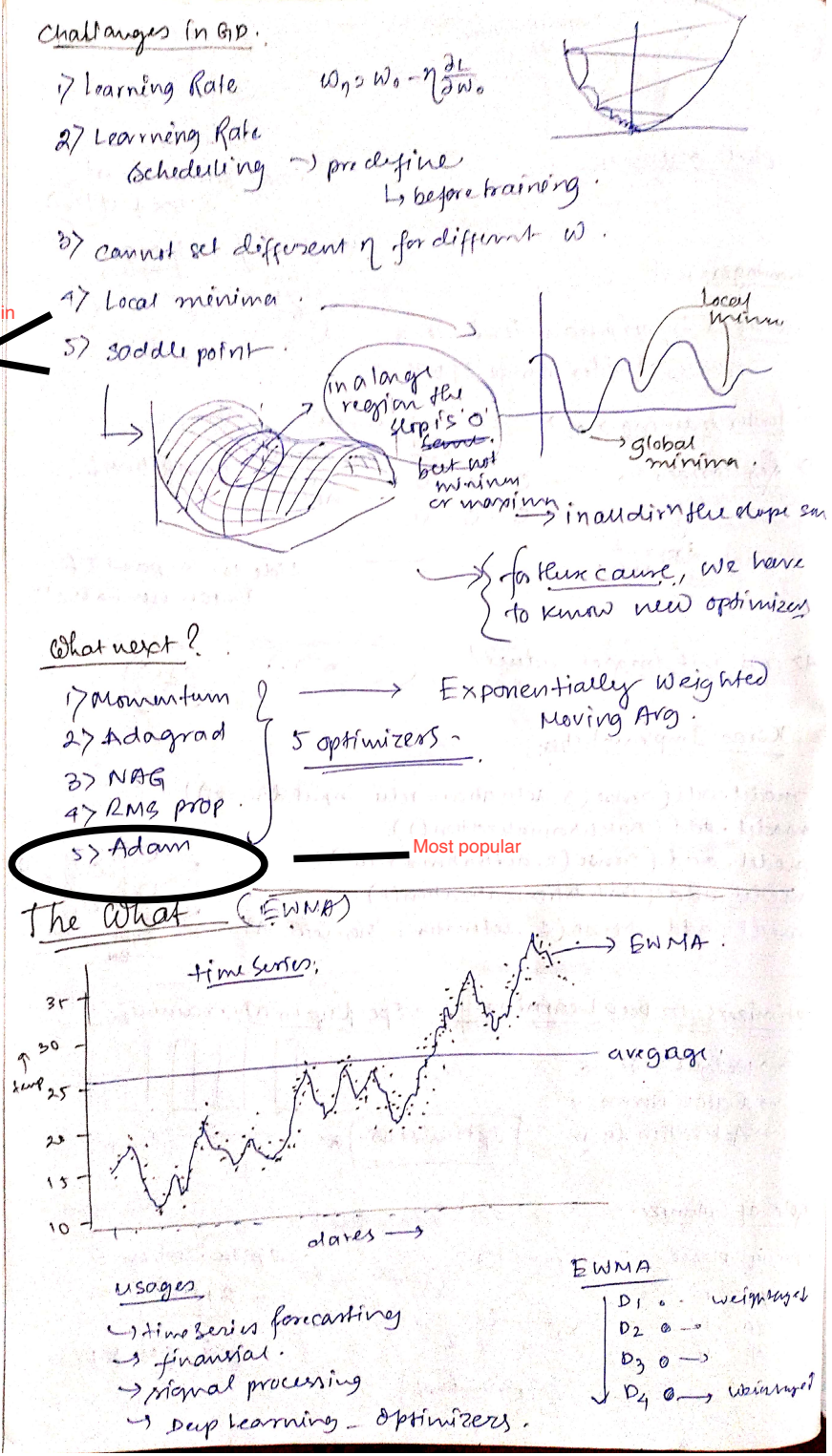






Scanned with OKEN Scanner

Mathematical formulation

$$V_t = \beta \cdot V_{t-1} + (1-\beta)\theta_t$$

$\beta \in [0, 1]$

$0 < \beta < 1$.

index	temp (θ)
D1	25
D2	13
D3	17
D4	31
D5	43

Suppose. $\beta = 0.9$

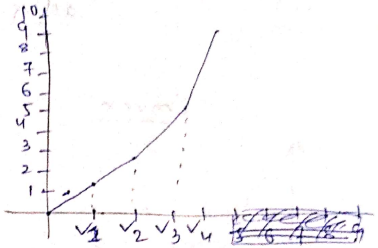
$V_0 = 0$ or $V_0 = \theta_0$

$V_1 = 0.9 \times V_0 + 0.1 \times \theta_1$
$= 0.9 \times 0 + 0.1 \times 13$
$= \boxed{1.3}$

$V_2 = 0.9 \times 1.3 + 0.1 \times 17$
$= \boxed{2.87}$

$V_3 = 0.9 \times 2.87 + 0.1 \times 31$
$= \boxed{5.68}$

$V_4 = 0.9 \times 5.68 + 0.1 \times 43$
$= \boxed{9.41}$



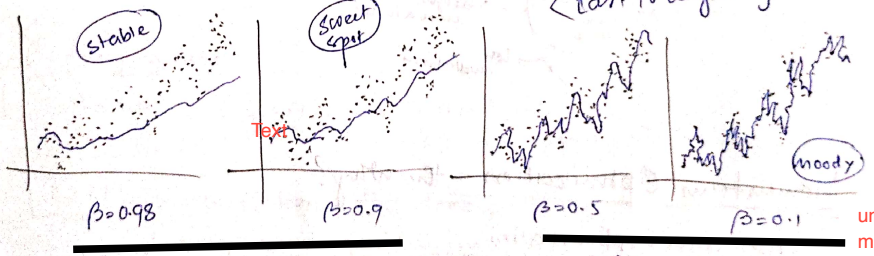
$\beta \rightarrow \boxed{\frac{1}{1-\beta}}$ days.

Let $\beta = 0.9 \rightarrow \frac{1}{1-0.9} = (10)$

↓

<last 10 days avg>

EWMA with different β



stable because wont much affected by present/current data

unstable,moody because more affected by present data

$\beta \uparrow$ → means. you are considering the weights of the old data significantly.

$\beta \downarrow$ → consider only just prev data or current data. graph will be spicy.

Mathematical Intuition

$V_t = \beta V_{t-1} + (1-\beta)\theta_t$.

$V_0 = 0$

$V_1 = (1-\beta)\theta_1$

$V_2 = \beta V_1 + (1-\beta)\theta_2$
$= \beta(1-\beta)\theta_1 + (1-\beta)\theta_2$

$V_3 = \beta V_2 + (1-\beta)\theta_3 = \beta^2(1-\beta)\theta_1 + \beta(1-\beta)\theta_2$

$V_4 = \beta V_3 + (1-\beta)\theta_4$
$= \beta^3(1-\beta)\theta_1 + \beta^2(1-\beta)\theta_2 + \beta(1-\beta)\theta_3 + (1-\beta)\theta_4$
$= (1-\beta)[\beta^3\theta_1 + \beta^2\theta_2 + \beta\theta_3 + \theta_4]$

$V_4 = (1-\beta)[\beta^3\theta_1 + \beta^2\theta_2 + \beta\theta_3 + \theta_4]$ $\quad \beta \in (0,1)$

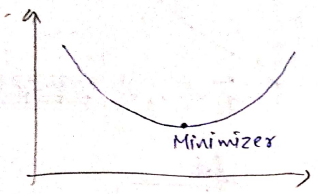
$\beta^3 < \beta^2 < \beta$ → less weightage, more weightage

pandas:

`df['col'].ewm(alpha=0.9).mean()`

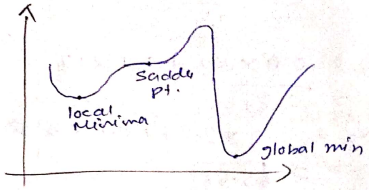
$\alpha = 1-\beta$. $\quad \alpha = 0.9 \rightarrow \beta = 0.1$

Convex



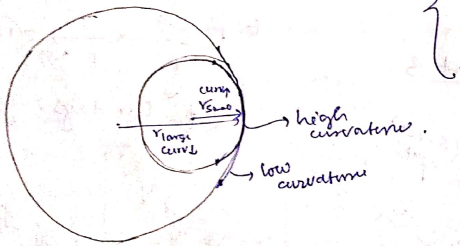
easily get minima.

Non convex



difficult to get minima.

1> local Minima
2> saddle pt.
3> high curvature.



Momentum Optimization – the Why?

Non-convex optimization

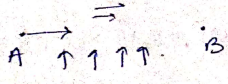
- high curvature
- Consistent gradient
- Noisy gradient (local minima)

momentum

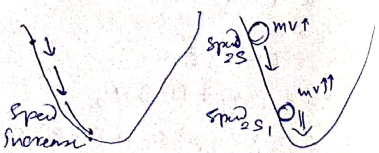
→ slope.

Momentum Optimization – The What?

Common



Newton



(mv) m → unit mass, velocity → Previous {history update}

$S_1 > S$.

Momentum Optimization – Mathematics (the how?)

$W_{t+1} = W_t - \eta \nabla W_t$.

acceleration in same direction

momentum ← history of past velocities

$v \rightarrow$ velocity

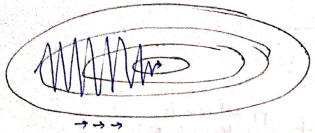
$W_{t+1} = W_t - v_t$.

$v_t = \beta \cdot v_{t-1} + \eta \nabla W_t$

$0 < \beta < 1$.

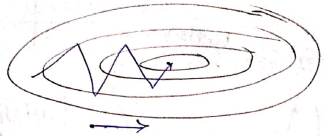
v_{t-2}, v_{t-3} — prev velocity.

at fist v=0
update happens in two steps



SGD

horizontal dirn movement ↓
vertical " " ↑



SGD momentum.

horizontal dirn movement ↑

Effect of beta (β).

$W_{t+1} = W_t - v_t$

$v_t = \beta v_{t-1} + \eta \nabla W_t$.

$\beta = 0$ → SGD → $W_{t+1} = W_t - \eta \nabla W_t$.

B=0.9 Recommended, good memory

β = decaying factor.

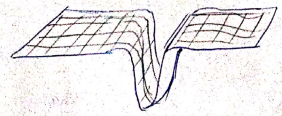
$\beta = 0.9 \rightarrow \frac{1}{1-\beta} = 10$ → current vel. is avg of past 10 velocities.

same concept as that of momentum

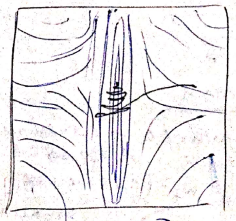
$\beta = 1$ → no friction → no decay
↳ dynamic equilibrium.

High B->long memory
low B->short memory

Problem with momentum optimization.



contour plot:



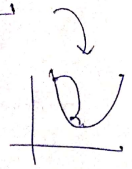
velocity gained & the minima pt. crossed due to the high velocity. & gradually comes to the minima after too much oscillations.

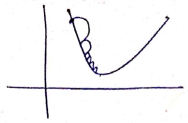
NAG – Nesterov Accelerated Gradient

momentum → $W_{t+1} = W_t - (\beta v_{t-1} + \eta \nabla W_t)$

past vel. ↑ gradient at that pt. ↑

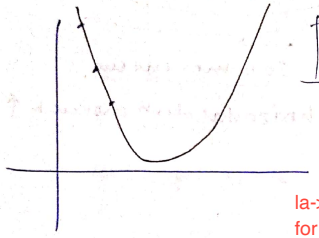
& normal → $W_{t+1} = W_t - \eta \nabla W_t$



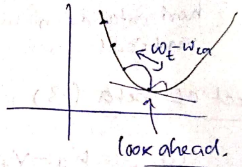


B more->more oscillation
Blow->less oscillation but slow

NAG better.



history of velocity + gradient at that pt.



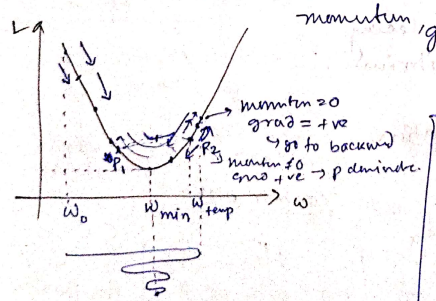
la->look ahead for correction

$W_{la} = W_t - \beta v_{t-1}$

$v_t = \beta v_{t-1} + \eta \cdot \nabla W_{la}$ → $v_t = (W_t - W_{la}) + \eta \nabla W_{la}$

$W_{t+1} = W_t - v_t$.

Geometric Intuition



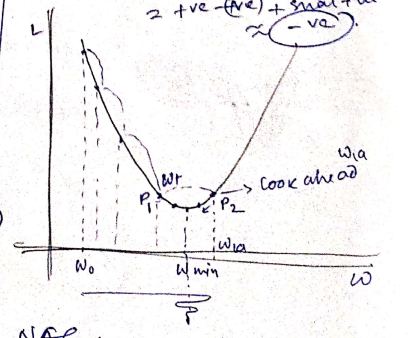
momentum, gradient.

-ve of grad/slope.

momentum

at P_2 → momentum ≠ 0 → very small +ve
∇W_t = +ve, at P_2 = 0.8
$\therefore W_{t+1} = W_t - (0.8 \times +ve + \eta \cdot +ve)$
small +ve
W_{t+1} = +ve

So, gradient small amount

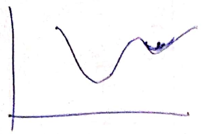


NAG ..

Scanned with OKEN Scanner

Disadvantage ①

NAG → oscillation → damped · local Minima



Keras Code · ~~Momentum~~

```
{tf.keras.optimizers.SGD(
    learning_rate = 0.01, momentum=0.0,
    nesterov = False, name = "SGD", **kwargs
)
```

{ NAG → momentum = 0.9 (β), nesterov = false | Momentum → momentum = 0.9 (β), nesterov = True. }

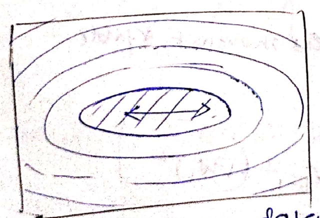
AdaGrad — Adaptive Gradient → changing gradient.

→ input features → scale ↳ different · ← { cgpa | salary | … ; 0 – 10 | 0 – lacs } ↓ Normalize.

→ features → sparse → most (0) ↳ Adagrad ← iq cgpa … (0, 0, 0, 0 … max 0.)

Problem

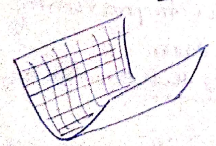
Elongated Bowl Problem



sparse data.

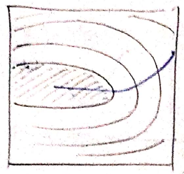
→ Normal

Normalized will be like a circle

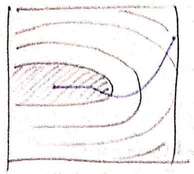


elongated ↓ slope → 1 axis → change, another → no change in slope.

How Optimizers behave (Why?)



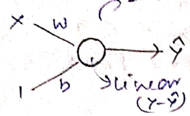
Batch GD



Momentum

m: b | x | y — 0.72, 0.81, 0, 0, 0, 0, 0, 0.2 | 0.3

less movement in the direction of 'x' b/c of most of 0's.



BGD → Sparse.

for i in epochs:

$\omega = \omega - \eta \frac{\partial L}{\partial w}$

$b = b - \eta \frac{\partial L}{\partial b}$ → not zero. → big number

[100 rows] → many zeros → small number, many time $\frac{\partial L}{\partial w} = 0$ → No update

$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$

$= -2(y-\hat{y}) \cdot x$ → add.

$\frac{\partial L}{\partial b} = -2(y-\hat{y}) * 1$

Adagrad Mathematics + Intuition

$\omega = w - \eta \frac{\partial L}{\partial w}$

$b = b - \eta \frac{\partial L}{\partial b}$

lr → small → gradient large ↑

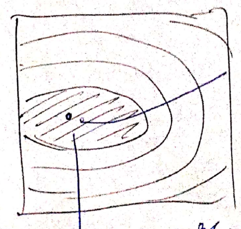
$\eta \uparrow \rightarrow \nabla L \downarrow$

$\eta \downarrow \rightarrow \nabla L \uparrow$ = to maintain the movement stable.

$$V_t = V_{t-1} + (\nabla W_t)^2$$

$$\omega_{t+1} = W_t - \frac{\eta \nabla W_t}{\sqrt{V_t + \epsilon}}$$

ϵ = very small no. ↓ it is given to prevent '0' division error. if $V_t = 0$, then also denominator ≠ 0.



Cannot converge.

Disadvantage

$\frac{\eta}{\sqrt{V_t}}$ → small / → Large } overall very very small. ↳ update stop

27/07/25 RMSProp — Root Mean Square Prop.

Improvement → AdaGrad ← Sparse data.

Adagrad. $V_t = V_{t-1} + (\nabla W_t)^2$

$W_{t+1} = W_t + \frac{\eta}{\sqrt{V_t + \epsilon}} \nabla W_t$.

→ sparse.

b	x	y
	0	
	0	
	0	
	0	
	72	
	0	
	0	

→ at the end $V_t \uparrow\uparrow$

So the $\frac{\eta}{\sqrt{V_t+\epsilon}} \downarrow\downarrow$ no update

So, cannot converge to the final minimum.

↓

RMS Prop.

$$V_t = \beta V_{t-1} + (1-\beta)(\nabla W_t)^2$$

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{V_t+\epsilon}} \nabla W_t \qquad (\beta = 0.95)$$

→ $V_0 = 0$

→ $V_1 = 0.95 \times 0 + 0.05 (\nabla W_1)^2$

$V_2 = 0.95 \times 0.05 (\nabla W_1)^2 + 0.05 (\nabla W_2)^2$ — epoch 2

$V_3 = [0.95 \times 0.95 \times 0.05] (\nabla W_1)^2 + 0.95 \times 0.05 (\nabla W_2)^2 + [0.05] (\nabla W_3)^2$

epoch 1 → smaller; epoch 3 → largest.

V_t short → λ. η → not become very small.
↳ can converge.

Adagrad → ~~NEA~~ LR. → ~~Non~~ convex optimization. (global minima)

↳ Problem NN. with non-convex → cannot converge.

but RMSProp → RLR, → NN } ✗

Disadvantage.

No, one of the best optimization technique.

Adam Optimizer → Adaptive Moment Optimizer

SGD / BGD / MBGD
→ Momentum → NAG
→ Adagrad
→ RMSProp.

→ 1) Momentum
2) learning decay } Merge

Mathematical Formulation

combines momentum with adaptive learning rate

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{V_t+\epsilon}} \times m_t$$

where $m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla W$ → Momentum

$V_t = \beta_2 V_{t-1} + (1-\beta_2)(\nabla W_t)^2$ → Adagrad

Bias Correction

epoch #1

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{V}_t = \frac{V_t}{1-\beta_2^t}$$

$\beta_1 = 0.9$, $\beta_2 = 0.99$ ↓ change parameters in keras.

Verdict

Adam ↓ RMSProp → Momentum.

~~28/07/25 CNN — Convolutional Neural Network~~

~~CNN also known as convnet, are a special kind of NN for processing data that has a known grid-like topology like time series data (1D) or images (2D).~~

~~ANN → Convolution~~

~~→ Convolution~~
~~→ Pooling~~
~~→ FC layer → ANN~~

~~ANN → Matrix Multiplication }~~

Scanned with OKEN Scanner